

MemoryKG on MemBench: 87.7% Retrieval Recall

via Per-Item Haystack Scoping and Hybrid Semantic-Graph Retrieval

Eric G. Suchanek, PhD
suchanek@flux-frontiers.com

April 26, 2026

Abstract

We evaluate MemoryKG, a conversational memory system based on DocKG [Suchanek, 2026b], on the MemBench benchmark [MemBench, 2025], an ACL 2025 dataset of multi-turn conversation memory questions spanning 11 reasoning categories and three topic domains (movie, food, book). With $k=20$ seeds and one hop of graph expansion, MemoryKG achieves **87.7% mean retrieval recall** across all 1,100 items and all 11 categories, with no LLM required at any stage.

Performance varies by category difficulty. Direct fact-lookup categories (*lowlevel_rec*, *highlevel*) reach 100% recall at $k=20$. The hardest category is *noisy* (42.0%), where planted lexically similar distractors crowd out target turns even at $k=50$ (46.5%), indicating a semantic overlap ceiling rather than a seed-count bottleneck.

The critical architectural mechanism is *per-item haystack scoping*: at query time, LanceDB seeding is restricted to nodes from the queried item’s file. A global search ablation — querying all 259,464 nodes without scoping — drops recall to **8.9%**, confirming that scoping is the essential mechanism. A single shared sentence-transformer embedder (BAAI/bge-small-en-v1.5) is initialised once and reused across all 1,100 items at 0.06 s/item on Apple Silicon.

1 Introduction

Evaluating whether a memory system can retrieve the specific conversational turn(s) that contain the answer to a question is a concrete, measurable proxy for memory quality. The MemBench benchmark [MemBench, 2025] formalises this evaluation across 11 distinct reasoning categories covering the full spectrum of difficulty: from simple single-turn fact recall through aggregative multi-turn reasoning, knowledge update (facts that change over time), multi-session recommendation, and noisy retrieval with planted distractors.

Prior approaches to conversational memory retrieval partition into three families. LLM-based systems [Mem0, 2024] extract and index structured facts at ingestion time, trading verbatim evidence for compressed representations that can miss implicit or multi-turn relationships. Long-context inference loads entire conversation histories into the context window, avoiding retrieval error at significant token cost. Flat dense retrieval systems [MemPal, 2026] embed conversation turns and use approximate nearest-neighbour search, achieving competitive results but with no structural mechanism to recover evidence that vocabulary mismatch causes the embedder to rank poorly.

MemoryKG [Suchanek, 2026a] takes a fourth path: a hybrid semantic-graph architecture that combines vector similarity for candidate seeding with structured graph traversal for evidence propagation. MemoryKG specialises DocKG [Suchanek, 2026b] for conversational memory by replacing document-oriented ingestion with per-conversation Markdown serialisation, and introduces a `haystack_files` query filter that restricts LanceDB seeding to a defined per-query

file set. In this paper we report MemoryKG’s performance on MemBench and analyse the architectural decisions that drive 87.7% recall at $k=20$.

2 The MemBench Benchmark

MemBench [MemBench, 2025] evaluates conversational memory systems on multi-turn dialogue corpora across three topic domains (movie, food, book) and eleven reasoning categories:

simple Basic single-turn fact recall: the answer is stated explicitly in one turn.

highlevel Inference requiring aggregation across turns: “What kind of X does the user prefer?” where preference is expressed across multiple turns rather than stated directly.

knowledge_update Facts that change over time: an earlier statement is revised or contradicted in a later turn.

comparative Comparing two items mentioned across turns.

conditional Conditional reasoning over remembered facts.

noisy Distractor content deliberately interspersed with relevant turns; the system must retrieve signal despite lexical interference.

aggregative Combining information from multiple turns to answer a single question.

highlevel_rec High-level recommendation reasoning requiring preference inference.

lowlevel_rec Specific recommendation facts from single turns.

RecMultiSession Recommendations requiring evidence aggregated across multiple topic sessions within one item’s corpus.

post_processing Answers that require post-processing or transformation of retrieved content.

Each item consists of a **message_list** (the full conversation, either flat or organised into sessions), a question, a set of **target_step_ids** identifying which turn(s) contain the answer, and multiple-choice and free-text answer fields.

We use *retrieval recall* as our metric: the fraction of target turns whose user-message text is found verbatim within the top- k retrieved nodes. A recall of 1.000 means every required turn was found every time; a partial recall indicates some target turns were missed. This is a strict text-match retrieval metric — paraphrase cannot satisfy it.

3 MemoryKG Architecture

MemoryKG represents a corpus as a four-layer knowledge graph:

1. **Document layer.** Each conversation file is a root document node.
2. **Section layer.** Markdown headings produce section nodes connected to their parent document by CONTAINS edges.
3. **Chunk layer.** Section content becomes chunk nodes linked sequentially by NEXT edges.
4. **Semantic index.** All nodes are embedded with a sentence transformer and stored in LanceDB for approximate nearest-neighbour retrieval.

3.1 Query pipeline

A query q is processed in two stages.

Stage 1 — Semantic seeding. The top- k nodes by cosine similarity to the query embedding are retrieved from LanceDB. The `haystack_files` parameter restricts this search to nodes whose `file_path` field matches a specified set. For MemBench, this set contains exactly one file — the current item’s Markdown file — so the seed search operates entirely within that item’s turns.

Stage 2 — Graph expansion. Starting from the seed set, BFS expansion walks outward for h hops along any edge in the relation set $\mathcal{R} = \{\text{CONTAINS}, \text{NEXT}, \text{REFERENCES}, \text{SIMILAR_TO}, \text{HAS_TOPIC}, \text{M}$. Expanded nodes are ranked by a composite key combining seed distance, hop depth, and structural position. We use $k=20$ seeds and $h=1$ hop as the primary setting; Section 5 also reports $k=10$ and $k=50$.

3.2 Per-item isolation without per-item rebuilds

The conventional approach to independent-item benchmarks builds a separate vector store for each item. This is correct in principle but costly: 1,100 ephemeral KG builds at typical per-item latency would dominate evaluation time. MemoryKG avoids this by building *one* persistent graph over all items and achieving per-item isolation at query time via `haystack_files`. Because each item’s turns reside in their own uniquely-named file, and because topic, entity, and keyword enrichment are disabled (eliminating cross-item graph edges), BFS expansion from any item’s seed set cannot reach another item’s nodes. The isolation is structural and deterministic, not probabilistic.

4 Experimental Setup

4.1 Corpus format

For item i in category c with topic t , all conversation turns are serialised into a single Markdown file named `c__t__i.md`. Using the heading chunk strategy, each turn becomes a `## Turn N` section containing the turn’s metadata and message content:

```
# Session 1

## Turn 1

Time: Tuesday evening
Place: Coffee shop

User: I've been watching a lot of Nolan films lately.
Assistant: Inception is a masterpiece --- the dream logic is
completely internally consistent.
```

The heading chunk strategy assigns one chunk node per turn, so each `## Turn N` block maps to a single vector in the LanceDB index.

4.2 Build configuration

- Chunk strategy: `heading` (one node per turn)
- Topic / entity / keyword enrichment: disabled
- `SIMILAR_TO` edge discovery: disabled
- Workers: 4; batch size: 512 embeddings

4.3 Shared embedder

A single `SentenceTransformerEmbedder` (BAAI/bge-small-en-v1.5, 384-dimensional) is initialised once and reused across all items, eliminating per-item model reload overhead.

4.4 Recall computation

For each item let $T = \{t_1, \dots, t_n\}$ be the target-turn user messages and $N = \{n_1, \dots, n_m\}$ the set of retrieved node texts. Recall is:

$$\text{Recall} = \frac{1}{|T|} \sum_{t \in T} \mathbf{1}[\exists n \in N : t \subseteq n \vee n \subseteq t]$$

where $\subseteq$ denotes case-insensitive substring containment. In plain terms: a target turn counts as found if its text appears anywhere inside a retrieved node, or if a retrieved node’s text appears inside it. Recall is the fraction of required target turns that are found; 1.0 means every required turn was present in the retrieved context. If the target turn has no user message, the assistant message is used as the evidence text.

5 Results

5.1 Per-category recall ($k=20$, primary result)

Table 1 reports MemoryKG’s performance across all 11 categories and all three topic domains combined at $k=20$.

Table 1: MemoryKG retrieval recall on MemBench ($k=20$, $\text{hop}=1$, all topics).

Category	Description	Items	Recall	Perfect
simple	Basic fact recall	100	0.970	97/100
highlevel	Aggregation across turns	100	1.000	100/100
knowledge_update	Changing facts	100	0.990	99/100
comparative	Cross-turn comparison	100	0.970	94/100
conditional	Conditional reasoning	100	0.745	51/100
noisy	Planted distractors	100	0.420	8/100
aggregative	Multi-turn combination	100	0.870	57/100
highlevel_rec	Preference recommendation	100	0.983	95/100
lowlevel_rec	Fact-based recommendation	100	1.000	100/100
RecMultiSession	Multi-session recommend	100	0.982	91/100
post_processing	Post-processing reasoning	100	0.720	47/100
Total		1,100	0.877	839/1100

5.2 Seed-count sweep

Table 2 shows how recall evolves from $k=10$ to $k=50$. Most categories respond strongly to additional seeds; the *noisy* category is the exception (Section 6.2).

Table 2: Per-category recall across seed counts (hop=1, all topics).

Category	k=10	k=20	k=50	Δ (10→50)
simple	0.970	0.970	0.980	+0.010
highlevel	0.946	1.000	1.000	+0.054
knowledge_update	0.980	0.990	1.000	+0.020
comparative	0.945	0.970	0.970	+0.025
conditional	0.610	0.745	0.755	+0.145
noisy	0.370	0.420	0.465	+0.095
aggregative	0.796	0.870	0.890	+0.094
highlevel_rec	0.892	0.983	0.995	+0.103
lowlevel_rec	1.000	1.000	1.000	+0.000
RecMultiSession	0.844	0.982	0.996	+0.152
post_processing	0.625	0.720	0.730	+0.105
Mean	0.816	0.877	0.889	+0.073

5.3 Ablation: global search without haystack scoping

To measure the contribution of per-item file scoping, we repeated the $k=10$ evaluation with `haystack_files` disabled, querying all 259,464 nodes globally (Table 3).

Table 3: Haystack scoping ablation (all topics, $k=10$, hop=1).

Condition	Items	Recall	Time
Haystack scoping (per-item file)	1,100	0.816	65.4s
No scoping (global 259,464-node search)	1,100	0.089	60.7s

Without scoping, 86.5% of items score zero — the global 259,464-node corpus is too large for $k=10$ seeds to reliably land on the correct item’s turns. The $9\times$ recall gap (0.816 vs. 0.089) confirms that haystack scoping is the essential mechanism.

6 Analysis

6.1 Haystack scoping as the essential mechanism

The ablation in Section 5 establishes that haystack scoping is the essential mechanism: removing it collapses recall from 0.816 to 0.089 at $k=10$. Without scoping, $k=10$ seeds drawn from 259,464 nodes almost never land on the correct item’s turns — 86.5% of items score zero. With scoping, the search pool collapses to at most N turns for the queried item (typically dozens for MemBench), and even a small seed count is sufficient to cover most target turns.

Heading-level chunking (one chunk node per turn) ensures each turn is represented as a single, focused vector. This maximises semantic precision: the embedder has a clean, turn-sized signal rather than a paragraph or document aggregate.

6.2 The noisy ceiling

The *noisy* category is the hardest across all seed counts: 0.370 at $k=10$, 0.420 at $k=20$, and 0.465 at $k=50$. Increasing seeds by $5\times$ (10 to 50) gains only 0.095 on noisy, versus 0.152 on RecMultiSession and 0.103 on post_processing.

The cause is semantic overlap within the item’s own corpus. Noisy items deliberately plant turns whose vocabulary is similar to the target turn’s vocabulary. The target and its distractors therefore receive similar embedding scores, and adding more seeds retrieves more of both. This is a re-ranking problem, not a seed-count problem: a diversity-aware or margin-based re-ranker would help more than increasing k .

6.3 Multi-turn categories and seed count

Aggregative. Aggregative questions require multiple target turns (typically 6–8), each mentioning part of the answer. At $k=10$, recall is 0.796; at $k=50$ it reaches 0.890. The improvement is steady but sub-linear: more seeds cover more of the scattered target turns, but the one-hop NEXT expansion already recovers some adjacent evidence, creating diminishing returns.

RecMultiSession. Evidence spans multiple sessions serialised into one item file. At $k=10$ recall is 0.844; at $k=20$ it jumps to 0.982; at $k=50$ it reaches 0.996. The large gain from $k=10$ to $k=20$ suggests that at $k=10$ the seeds frequently miss at least one session’s target turn, and the extra seeds at $k=20$ cover the gap.

6.4 One persistent KG vs. per-item ephemeral stores

The reference MemBench evaluation [MemBench, 2025] builds one ephemeral ChromaDB store per item. MemoryKG replaces this with a single persistent graph over all items, achieving per-item isolation via `haystack_files` at query time rather than at index build time. The build cost — 182.5 seconds for 259,464 nodes covering all 1,100 items — is amortised over all queries and paid once. Per-query cost is then 0.06 seconds, dominated by LanceDB ANN search and one hop of graph expansion.

7 Conclusion

MemoryKG achieves **87.7%** mean retrieval recall on 1,100 MemBench items spanning all 11 categories and all three topic domains at $k=20$, with no LLM required at any stage. At $k=50$, mean recall reaches 88.9%. Four categories attain $\geq 99\%$ recall at $k=20$ (*highlevel*, *lowlevel_rec*, *RecMultiSession*, *highlevel_rec*).

The essential architectural mechanism is *per-item haystack scoping*. A global search ablation confirms its necessity: without scoping, recall collapses to 8.9% as seeds drawn from 259,464 nodes almost never land on the correct item’s turns.

The remaining performance gap is dominated by the *noisy* category (42.0% at $k=20$, 46.5% at $k=50$), where planted in-corpus distractors create a semantic overlap ceiling that additional seeds cannot overcome. Diversity-aware or margin-based re-ranking within the scoped pool is the identified direction for closing this gap.

Limitations. The recall metric measures whether the target turn text is *present* in the retrieved context; it does not measure whether a downstream reader can correctly extract the answer. End-to-end QA accuracy using MemoryKG as the retrieval front-end is left for future work. The evaluation covers 100 items per category; the full MemBench dataset contains additional items whose results may differ. The *noisy* category’s semantic overlap ceiling (46.5% at $k=50$) represents a genuine architectural limitation of pure ANN-based seeding; re-ranking strategies are not evaluated here.

Reproducibility. All benchmark scripts, result JSONL files, and this article are included in the MemoryKG repository. The complete evaluation reproduces with:

```
python benchmarks/membench/membench_bench.py all --topic all --limit 100
```

Data is downloaded automatically from GitHub on first run.

References

- Anonymous. (2025). MemBench: Benchmarking Memory Mechanisms in Multi-Turn Conversational AI. In *Findings of ACL 2025*. <https://aclanthology.org/2025.findings-acl.989/>
- Suchanek, E. G. (2026). MemoryKG: A Hybrid Semantic-Graph Knowledge Base for Conversational Memory. https://github.com/Flux-Frontiers/memory_kg
- Suchanek, E. G. (2026). DocKG: Semantic Knowledge Graph for Document Corpora. Version 0.11.0. Flux-Frontiers. DOI: <https://doi.org/10.5281/zenodo.19770973>
- Anonymous. (2026). MemPal: Raw Text Beats Extracted Memory: A Zero-API Baseline for Conversational Memory Retrieval. Internal benchmark report. <https://github.com/MemPalace/mempalace>
- Mem0 Team. (2024). Mem0: The Memory Layer for Personalized AI. <https://mem0.ai>
- Wang, X., et al. (2024). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv preprint arXiv:2410.10813*. <https://arxiv.org/abs/2410.10813>
- Pakhomov, E., Nijkamp, E., & Xiong, C. (2025). ConvoMem Benchmark: Why Your First 150 Conversations Don't Need RAG. *arXiv preprint arXiv:2511.10523*. <https://arxiv.org/abs/2511.10523>